```
"Combatant.h"
---------
#pragma once

#include <Urho3D/Scene/Component.h>
#include <Urho3D/Graphics/AnimationController.h>
#include <Urho3D/Scene/Node.h>
#include <Urho3D/Core/StringUtils.h>

using namespace Urho3D;

enum SIDE { SIDE_HERO = 0, SIDE_VILLAIN = 1 };

class Combatant : public Component
{
    URHO3D_OBJECT(Combatant, Component);

public:
    Combatant(Context* context)
        : Component(context)
        , node_(nullptr)
        , animController_(nullptr)
        , isFrozen_(false)
        , freezeTimer_(0.0f)
        , side_(0)
        , maxHealth_(100)
        , health_(100)
    {}

    virtual void Initialize(Node* node, const ea::string& prefabPath) = 0;
    virtual void Update(float timeStep) = 0;
    virtual bool CheckCollision(Combatant* other) = 0;
    virtual void HandleHit(Combatant* attacker) = 0;

    // Make SetFrozen VIRTUAL so Zombie can override it
    virtual void SetFrozen(bool freeze, bool lockAnimation = true) = 0;

    virtual void Reset() = 0;

    // Common utility accessors
    virtual float GetForwardOffset() const { return 0.0f; } // Added for lunge tracking

    Node* GetNode() const { return node_; }
    Vector3 GetPosition() const { return node_ ? node_->GetWorldPosition() : Vector3::ZERO; }
    void SetPosition(const Vector3& pos) { if (node_) node_->SetWorldPosition(pos); }

    bool IsFrozen() const { return isFrozen_; }

protected:
    Node* node_;
    AnimationController* animController_;
    bool isFrozen_;
    float freezeTimer_;
    int side_;
    int maxHealth_;
    int health_;
};
"CombatSystem.cpp"
---------
#include "CombatSystem.h"
#include <Urho3D/IO/Log.h>
#include <Urho3D/Resource/ResourceCache.h>
#include <Urho3D/Scene/Scene.h>
#include <Urho3D/Graphics/AnimationController.h>
#include <Urho3D/Physics/RigidBody.h>
#include <Urho3D/Graphics/StaticModel.h>
#include <Urho3D/Graphics/Model.h>
#include <Urho3D/Graphics/Material.h>
#include <Urho3D/Graphics/Technique.h>
#include <Urho3D/Graphics/AnimatedModel.h>
#include <Urho3D/Graphics/CustomGeometry.h>
#include <cmath>

CombatSystem::CombatSystem(Context* context)
    : Component(context)
    , fighter_(nullptr)
    , zombie_(nullptr)
    , debugText_(nullptr)
    , arenaHitFrozen_(false)
    , biteDelayTimer_(0.0f)
    , biteDelayActive_(false)
    , bloodScale_(1.0f)
    , bloodInitialized_(false)
    , bloodComputeDelay_(0.0f)
{
    // FINAL FIXED CODES CALIBRATED FROM YOUR ACCEPTED REVENUE TWEAK LOGS
    zombieOffsetScale_ = 0.820f;
    zombieOffsetRotation_ = 78.0f;
    fighterOffsetRotation_ = 0.0f;
    zombieOffsetPosition_ = Vector3(-0.865f, 0.0f, 0.000f); // Final verified gap offset metric boundary
    fighterOffsetPosition_ = Vector3(0.000f, 0.0f, 0.000f);
}

void CombatSystem::Initialize(Fighter* fighter, Zombie* zombie, Text* uiText)
{
    fighter_ = fighter;
    zombie_ = zombie;
    debugText_ = uiText;
    arenaHitFrozen_ = false;
    biteDelayTimer_ = 0.0f;
    biteDelayActive_ = false;
    bloodScale_ = 1.0f;
    bloodInitialized_ = false;
    bloodComputeDelay_ = 0.0f;
    ClearBloodWounds();
}

// Helper: is a point (in fighter-local space) inside the fighter capsule?
static bool IsInsideFighterCapsule(const Vector3& p)
{
    const float radius = 0.45f;      // capsule radius
    const float hHalf  = 0.80f;      // half of 1.6 height
    const Vector3 c(0.0f, 0.80f, 0.0f);

    float dy = p.y_ - c.y_;
    if (Abs(dy) <= hHalf)
    {
        // Cylinder section
        return Sqrt(p.x_ * p.x_ + p.z_ * p.z_) <= radius;
    }
    else
    {
        // Spherical end-cap
        Vector3 capCenter(0.0f, (dy > 0.0f) ? 1.6f : 0.0f, 0.0f);
        return (p - capCenter).Length() <= radius;
    }
}

void CombatSystem::SpawnBloodDisc(const Vector3& meshLocalPos, const Vector3& meshLocalNormal)
{
    Node* parent = fighter_ ? fighter_->GetNode() : nullptr;
    if (!parent) return;

    // Find the actual mesh node where the visual model lives
    Node* meshNode = parent->GetChild("AxeManMesh");
    if (!meshNode) meshNode = parent;

    Node* wound = meshNode->CreateChild("BloodDisc");
    // Push 3 cm out along the surface normal so it sits ON the skin
    wound->SetPosition(meshLocalPos + meshLocalNormal * 0.03f);
    // Rotate so local Y aligns with the outward normal
    wound->SetRotation(Quaternion(Vector3::UP, meshLocalNormal));

    auto* cache = GetSubsystem<ResourceCache>();
    auto* sm = wound->CreateComponent<StaticModel>();
    sm->SetModel(cache->GetResource<Model>("Models/Box.mdl"));

    Material* mat = new Material(context_);
    mat->SetTechnique(0, cache->GetResource<Technique>("Techniques/NoTexture.xml"));
    mat->SetShaderParameter("MatDiffColor", Variant(Color(1.0f, 0.0f, 0.0f))); // BRIGHT RED
    sm->SetMaterial(mat);

    // Flat disc: wide in X/Z, paper-thin along the normal (Y)
    wound->SetScale(Vector3(0.10f, 0.005f, 0.10f) * bloodScale_);

    bloodWounds_.push_back(SharedPtr<Node>(wound));

    URHO3D_LOGINFOF("[BLOOD] >>> SPAWNED disc on MESH at local=%s  normal=%s  scale=%.2f",
                    meshLocalPos.ToString().c_str(),
                    meshLocalNormal.ToString().c_str(),
                    bloodScale_);
}

void CombatSystem::GrowBloodWounds(float delta)
{
    bloodScale_ = Clamp(bloodScale_ + delta, 0.2f, 8.0f);
    URHO3D_LOGINFOF("[BLOOD] Grow/Shrink: scale=%.2f  count=%zu", bloodScale_, bloodWounds_.size());
    for (auto& w : bloodWounds_)
    {
        if (w) w->SetScale(Vector3(0.10f, 0.005f, 0.10f) * bloodScale_);
    }
}

void CombatSystem::ClearBloodWounds()
{
    URHO3D_LOGINFOF("[BLOOD] Clearing %zu old wounds", bloodWounds_.size());
    for (auto& w : bloodWounds_)
    {
        if (w) w->Remove();
    }
    bloodWounds_.clear();
    bloodScale_ = 1.0f;
    bloodInitialized_ = false;
    bloodComputeDelay_ = 0.0f;
}

void CombatSystem::ComputeBloodIntersections()
{
    ClearBloodWounds();
    URHO3D_LOGINFO("[BLOOD] ========== ComputeBloodIntersections START ==========");

    Node* fNode = fighter_->GetNode();
    Node* zNode = zombie_->GetNode();
    if (!fNode || !zNode)
    {
        URHO3D_LOGWARNING("[BLOOD] Missing fighter or zombie node!");
        return;
    }

    // ---- Find fighter mesh node (where wounds should attach) ----
    Node* fMesh = fNode->GetChild("AxeManMesh");
    if (!fMesh) fMesh = fNode;
    AnimatedModel* fAm = fMesh->FindComponent<AnimatedModel>();
    if (!fAm) fAm = fNode->FindComponent<AnimatedModel>();
    if (!fAm)
    {
        URHO3D_LOGWARNING("[BLOOD] Fighter has no AnimatedModel!");
    }

    // ---- Get zombie skeleton ----
    Node* zMesh = zNode->GetChild("ZombieMesh");
    if (!zMesh) zMesh = zNode;
    AnimatedModel* zAm = zMesh->FindComponent<AnimatedModel>();
    if (!zAm) zAm = zNode->FindComponent<AnimatedModel>();
    if (!zAm)
    {
        URHO3D_LOGWARNING("[BLOOD] Zombie has no AnimatedModel! Using fallback.");
    }

    bool useFallback = true;
    int tested = 0, passed = 0;

    if (zAm)
    {
        Skeleton& zSkel = zAm->GetSkeleton();
        unsigned numBones = zSkel.GetNumBones();
        URHO3D_LOGINFOF("[BLOOD] Zombie skeleton bones: %u", numBones);

        ea::vector<Vector3> spawnedLocals;
        const float minDist = 0.06f;

        for (unsigned i = 0; i < numBones && bloodWounds_.size() < 24; ++i)
        {
            Bone* bone = zSkel.GetBone(i);
            if (!bone || !bone->node_) continue;

            ea::string bName = bone->name_;
            bool relevant = false;
            if (bName.find("Spine")  != ea::string::npos) relevant = true;
            if (bName.find("Neck")   != ea::string::npos) relevant = true;
            if (bName.find("Clavicle") != ea::string::npos) relevant = true;
            if (bName.find("Shoulder") != ea::string::npos) relevant = true;
            if (bName.find("Arm")    != ea::string::npos) relevant = true;
            if (bName.find("Hand")   != ea::string::npos) relevant = true;
            if (bName.find("Head")   != ea::string::npos) relevant = true;
            if (bName.find("Chest")  != ea::string::npos) relevant = true;
            if (bName.find("Torso")  != ea::string::npos) relevant = true;
            if (!relevant) continue;

            tested++;
            Vector3 bWorld = bone->node_->GetWorldPosition();

            // CRITICAL: Convert to FIGHTER MESH local space, not pivot space
            Vector3 bLocal = fMesh->WorldToLocal(bWorld);

            // The mesh is scaled 5/9 and rotated, so the capsule shape is different in mesh space
            // Approximate: mesh scale is 5/9 = 0.555, so expand test radius accordingly
            const float meshScale = 5.0f / 9.0f;
            const float radius = 0.45f / meshScale;   // ~0.81 in mesh local units
            const float hHalf  = 0.80f / meshScale;   // ~1.44

            // Only the left side (negative X in mesh space, where zombie merges)
            if (bLocal.x_ > 0.05f / meshScale) continue;

            // Capsule test in mesh local space
            Vector3 toCentre = bLocal - Vector3(0.0f, 0.8f / meshScale, 0.0f);
            toCentre.y_ = 0.0f;
            if (toCentre.Length() > radius + 0.1f) continue;
            if (bLocal.y_ < -0.2f / meshScale || bLocal.y_ > 1.8f / meshScale) continue;

            // Deduplicate
            bool tooClose = false;
            for (const Vector3& ex : spawnedLocals)
            {
                if ((ex - bLocal).Length() < minDist / meshScale) { tooClose = true; break; }
            }
            if (tooClose) continue;

            Vector3 normal = bLocal - Vector3(0.0f, 0.8f / meshScale, 0.0f);
            normal.y_ = 0.0f;
            if (normal.Length() < 0.001f) normal = Vector3(-1.0f, 0.0f, 0.0f);
            else normal.Normalize();

            SpawnBloodDisc(bLocal, normal);
            spawnedLocals.push_back(bLocal);
            passed++;
        }

        URHO3D_LOGINFOF("[BLOOD] Bone result: %d tested, %d wounds spawned", tested, passed);
        if (!bloodWounds_.empty()) useFallback = false;
    }
    else
    {
        URHO3D_LOGWARNING("[BLOOD] Zombie has no AnimatedModel!");
    }

    if (useFallback)
    {
        URHO3D_LOGWARNING("[BLOOD] No bone wounds! Using fallback ring on MESH.");
        const float meshScale = 5.0f / 9.0f;
        const Vector3 c(0.0f, 0.8f / meshScale, 0.0f);
        const float rad = 0.45f / meshScale;

        for (int i = 0; i < 8; ++i)
        {
            float a = (float)i / 8.0f * 360.0f * 3.14159265f / 180.0f;
            Vector3 pt(-rad, c.y_ + 0.25f / meshScale * Sin(a), 0.25f / meshScale * Cos(a));
            Vector3 n = (pt - c).Normalized();
            if (n.Length() < 0.001f) n = Vector3(-1,0,0);
            SpawnBloodDisc(pt, n);
        }
        SpawnBloodDisc(Vector3(-rad, 0.9f / meshScale, 0.0f), Vector3(-1,0,0));
    }

    URHO3D_LOGINFOF("[BLOOD] ========== END: total wounds = %zu ==========", bloodWounds_.size());
}

void CombatSystem::UpdateCombat(float timeStep)
{
    if (!fighter_ || !zombie_) return;

    // 1. Regular approach update logic
    if (!arenaHitFrozen_ && !zombie_->IsNeckBiting())
    {
        zombie_->CheckCollision(fighter_);
    }

    // ---- 2-second contact timer (PAUSED while fighter is mid-swing) ----
    if (!arenaHitFrozen_ && zombie_->IsContactFrozen() &&
        !zombie_->IsNeckBiting() && !zombie_->IsFrozen())
    {
        if (!biteDelayActive_)
        {
            biteDelayActive_ = true;
            biteDelayTimer_ = 0.0f;
        }
        // ONLY tick when the fighter is NOT chopping
        if (!fighter_->IsMidSwing())
        {
            biteDelayTimer_ += timeStep;
        }
    }
    else
    {
        biteDelayActive_ = false;
        biteDelayTimer_ = 0.0f;
    }

    // 2. Weapon slice confirmation trace
    // CHANGED: removed IsContactFrozen requirement.
    // The fighter can land a hit as long as the zombie isn't already frozen
    // and the neckbite hasn't started. Distance check still keeps it fair.
    if (fighter_->IsMidSwing() && !zombie_->IsFrozen() &&
        !zombie_->IsNeckBiting() && !arenaHitFrozen_)
    {
        float deltaX = std::abs(fighter_->GetPosition().x_ - zombie_->GetPosition().x_);
        if (deltaX < 1.4f)
        {
            arenaHitFrozen_ = true;
            biteDelayActive_ = false;   // cancel pending neckbite
            biteDelayTimer_ = 0.0f;
            zombie_->HandleHit(fighter_);
            if (debugText_)
            {
                debugText_->SetText("CHOP DETECTED! PRESS SPACEBAR TO SPAWN NEXT RUNNER.");
                debugText_->SetColor(Color::RED);
            }
            return;
        }
    }

    // 3. Perfect Sync Bite Sequence — DISABLED FOR OFFSET TUNING
    if (NECKBITE_DEBUG_ENABLED && biteDelayActive_ && biteDelayTimer_ > 2.0f && !fighter_->IsMidSwing())
    {
        biteDelayActive_ = false;
        biteDelayTimer_ = 0.0f;
        arenaHitFrozen_ = true;

        zombie_->StartNeckBite(fighter_->GetNode());
        zombie_->SetBiteSynced(true);
        fighter_->SetBiteSynced(true);

        // ---- BLOOD: deferred until spatial-merge block (0.4 s delay) ----
        ClearBloodWounds();

        if (debugText_)
        {
            debugText_->SetText("NECK BITE SEQUENCING ACTIVE.");
            debugText_->SetColor(Color::YELLOW);
        }

        Node* fNode = fighter_->GetNode();
        Node* zNode = zombie_->GetNode();

        if (fNode && zNode)
        {
            // KEEP DYNAMIC COLLIDERS ACTIVE: Enforce capsule separation walls safely
            if (fighter_->GetRigidBody()) {
                fighter_->GetRigidBody()->SetKinematic(true);   // was false
                fighter_->GetRigidBody()->SetLinearVelocity(Vector3::ZERO);
                fighter_->GetRigidBody()->SetAngularVelocity(Vector3::ZERO);
            }
            if (zombie_->GetRigidBody()) {
                zombie_->GetRigidBody()->SetKinematic(true);    // was false
                zombie_->GetRigidBody()->SetLinearVelocity(Vector3::ZERO);
                zombie_->GetRigidBody()->SetAngularVelocity(Vector3::ZERO);
            }

            // Apply production orientation coordinates
            fNode->SetRotation(Quaternion(0.0f, fighterOffsetRotation_, 0.0f));
            zNode->SetRotation(Quaternion(0.0f, zombieOffsetRotation_, 0.0f));

            Vector3 currentFighterPos = fNode->GetPosition();
            currentFighterPos.y_ = 0.0f;
            fNode->SetPosition(currentFighterPos);

            Vector3 targetZombiePos = currentFighterPos + zombieOffsetPosition_;
            zNode->SetPosition(targetZombiePos);

            if (fighter_->GetRigidBody()) fighter_->GetRigidBody()->SetPosition(currentFighterPos);
            if (zombie_->GetRigidBody()) zombie_->GetRigidBody()->SetPosition(targetZombiePos);

            // Run synchronized track branches recursively via children lookups
            AnimationController* fAnim = nullptr;
            ea::vector<Node*> fChildren;
            fNode->GetChildrenWithComponent<AnimationController>(fChildren, true);
            if (auto* topF = fNode->GetComponent<AnimationController>()) fAnim = topF;
            else if (!fChildren.empty() && fChildren.front()) fAnim = fChildren.front()->GetComponent<AnimationController>();

            if (fAnim)
            {
                ea::string path = "fighter-nek-bite/test.glb.d/Animations/Armature.001_Armature.001_mixamo.com_Layer0.ani";
                fAnim->StopAll();
                AnimationParameters params(context_, path);
                params.looped_ = true;
                fAnim->PlayNewExclusive(params);
            }

            AnimationController* zAnim = nullptr;
            ea::vector<Node*> zChildren;
            zNode->GetChildrenWithComponent<AnimationController>(zChildren, true);
            if (auto* topZ = zNode->GetComponent<AnimationController>()) zAnim = topZ;
            else if (!zChildren.empty() && zChildren.front()) zAnim = zChildren.front()->GetComponent<AnimationController>();

            if (zAnim)
            {
                ea::string path = "zombie-nek-bite/test.glb.d/Animations/Armature_Armature_mixamo.com_Layer0_0.ani";
                zAnim->StopAll();
                AnimationParameters params(context_, path);
                params.looped_ = true;
                zAnim->PlayNewExclusive(params);
            }
        }
    }

    // =========================================================================
    // PERMANENT SPATIAL MERGE ENFORCEMENT FRAME CONTROL
    // =========================================================================
    if (arenaHitFrozen_ && zombie_->IsNeckBiting())
    {
        // ---- BLOOD: one-shot after 0.4 s delay ----
        if (!bloodInitialized_)
        {
            bloodComputeDelay_ += timeStep;
            URHO3D_LOGINFOF("[BLOOD] Delay: %.2f / 0.40", bloodComputeDelay_);
            if (bloodComputeDelay_ > 0.40f)
            {
                ComputeBloodIntersections();
                bloodInitialized_ = true;
                bloodComputeDelay_ = 0.0f;
            }
        }

        Node* fNode = fighter_->GetNode();
        Node* zNode = zombie_->GetNode();

        if (fNode && zNode)
        {
            // Lock orientations perfectly face-to-face on every single frame loop update
            fNode->SetRotation(Quaternion(0.0f, fighterOffsetRotation_, 0.0f));
            zNode->SetRotation(Quaternion(0.0f, zombieOffsetRotation_, 0.0f));

            // Snap fighter to floor baseline coordinates
            Vector3 fPos = fNode->GetPosition();
            fPos.y_ = 0.0f;
            fNode->SetPosition(fPos);

            // FORCE ENFORCED MERGE SEPARATION: Explicit vector lock blocks body merging completely
            Vector3 zPos = fPos + zombieOffsetPosition_;
            zNode->SetPosition(zPos);

            // Hard sync the physical hulls to stop gravity drops or sliding collisions
            if (fighter_->GetRigidBody()) {
                fighter_->GetRigidBody()->SetKinematic(true);   // enforce authority
                fighter_->GetRigidBody()->SetPosition(fPos);
                fighter_->GetRigidBody()->SetLinearVelocity(Vector3::ZERO);
                fighter_->GetRigidBody()->SetAngularVelocity(Vector3::ZERO);
            }
            if (zombie_->GetRigidBody()) {
                zombie_->GetRigidBody()->SetKinematic(true);    // enforce authority
                zombie_->GetRigidBody()->SetPosition(zPos);
                zombie_->GetRigidBody()->SetLinearVelocity(Vector3::ZERO);
                zombie_->GetRigidBody()->SetAngularVelocity(Vector3::ZERO);
            }
        }
    }
}

void CombatSystem::HandleManualAttack()
{
    if (!fighter_ || !zombie_) return;
    if (zombie_->IsFrozen() || zombie_->IsNeckBiting())
    {
        URHO3D_LOGINFO("[SYSTEM-INPUT] SPACEBAR: Resetting arena stage...");
        ResetSimulation();
        return;
    }

    // Compute lunge target: step in so fighter capsule overlaps zombie capsule by ~1 cm
    Vector3 fPos = fighter_->GetPosition();
    Vector3 zPos = zombie_->GetPosition();
    Vector3 delta = zPos - fPos;
    delta.y_ = 0.0f;
    float dist = delta.Length();

    const float overlap   = 0.01f;   // 1 cm penetration
    const float fRadius   = 0.45f;   // fighter capsule radius
    const float zRadius   = 0.45f;   // zombie capsule radius
    float step = dist - fRadius - zRadius + overlap;

    if (step < 0.0f) step = 0.0f;    // already close enough
    if (step > 0.35f) step = 0.35f;// safety cap: never lunge more than 35 cm

    Vector3 lungeGoal = fPos + delta.Normalized() * step;
    lungeGoal.y_ = fPos.y_;          // keep floor Y exactly
    fighter_->SetLungeTarget(lungeGoal);

    URHO3D_LOGINFO("[SYSTEM-INPUT] SPACEBAR: Executing fighter axe swing command.");
    fighter_->StartAttack();
}

void CombatSystem::ResetSimulation()
{
    ClearBloodWounds();   // <-- ADD
    arenaHitFrozen_ = false;
    biteDelayTimer_ = 0.0f;
    biteDelayActive_ = false;
    if (fighter_) fighter_->Reset();
    if (zombie_) zombie_->Reset();
    if (debugText_)
    {
        debugText_->SetText("STAGE ACTIVE: RUNNING...");
        debugText_->SetColor(Color::GREEN);
    }
}
"CombatSystem.h"
---------
#pragma once
#include <Urho3D/Scene/Component.h>
#include <Urho3D/UI/Text.h>
#include <Urho3D/Graphics/StaticModel.h>
#include <Urho3D/Graphics/Model.h>
#include <Urho3D/Graphics/Material.h>
#include <Urho3D/Graphics/Technique.h>
#include <Urho3D/Graphics/AnimatedModel.h>
#include <Urho3D/Graphics/CustomGeometry.h>
#include "Fighter.h"
#include "Zombie.h"

using namespace Urho3D;

class CombatSystem : public Component
{
    URHO3D_OBJECT(CombatSystem, Component);

public:
     CombatSystem(Context* context);
     ~CombatSystem() override = default;

     void Initialize(Fighter* fighter, Zombie* zombie, Text* uiText);
     void UpdateCombat(float timeStep);
     void HandleManualAttack();
     void ResetSimulation();

     void ComputeBloodIntersections();
     void SpawnBloodDisc(const Vector3& localPos, const Vector3& localNormal);
     void GrowBloodWounds(float delta);
     void ClearBloodWounds();

     bool IsArenaHitFrozen() const { return arenaHitFrozen_; }

     static constexpr bool NECKBITE_DEBUG_ENABLED = true;    // re-enabled

private:
     Fighter* fighter_;
     Zombie* zombie_;
     Text* debugText_;
     bool arenaHitFrozen_;
     float biteDelayTimer_ = 0.0f;
     bool  biteDelayActive_ = false;

    float zombieOffsetScale_;
    float zombieOffsetRotation_;
    float fighterOffsetRotation_;
    Vector3 zombieOffsetPosition_;
    Vector3 fighterOffsetPosition_;

    ea::vector<SharedPtr<Node>> bloodWounds_;
    float bloodScale_ = 1.0f;
    bool  bloodInitialized_ = false;
    float bloodComputeDelay_ = 0.0f;
};

"Fighter.cpp"
---------
#include "Fighter.h"
#include <Urho3D/IO/Log.h>
#include <Urho3D/Core/Context.h>
#include <Urho3D/Resource/ResourceCache.h>
#include <Urho3D/Scene/PrefabResource.h>
#include <Urho3D/Graphics/AnimatedModel.h>
#include <cstring>

// Physics collision layers
enum CollisionLayer {
    LAYER_WORLD = 1,
    LAYER_FIGHTER = 2,
    LAYER_ZOMBIE = 4
};

Fighter::Fighter(Context* context) : LogicComponent(context)
{
    SetUpdateEventMask(USE_FIXEDUPDATE);
}

void Fighter::RegisterObject(Context* context)
{
    context->RegisterFactory<Fighter>();
}

void Fighter::Initialize(Node* node, const ea::string& prefabPath)
{
    node_ = node;
    idleAnimPath_.clear();
    attackAnimPath_ = "test.glb.d/Animations/Armature_Armature_mixamo.com_Layer0.ani";

    if (node_)
    {
        arenaPosition_ = node_->GetPosition();
        auto* cache = GetSubsystem<ResourceCache>();
        if (cache && !prefabPath.empty())
        {
            auto* prefabFile = cache->GetResource<PrefabResource>(prefabPath);
            if (prefabFile)
            {
                meshNode_ = node_->CreateChild("AxeManMesh");
                prefabFile->InstantiateReference(meshNode_);
                meshNode_->SetRotation(Quaternion(0.0f, -90.0f, 0.0f));
                meshNode_->SetScale(5.0f / 9.0f);

                Vector3 pos = meshNode_->GetPosition();
                pos.y_ = 0.05f;
                meshNode_->SetPosition(pos);

                animController_ = meshNode_->FindComponent<AnimationController>();
                if (animController_) animController_->StopAll();
            }
        }

        if (meshNode_) ReparentWeapon(meshNode_);
        SetupPhysics();
    }
    Reset();
}

void Fighter::Initialize(Node* node, const ea::string& idleAnimPath, const ea::string& attackAnimPath)
{
    node_ = node;
    idleAnimPath_ = idleAnimPath;
    attackAnimPath_ = attackAnimPath;
    if (node_)
    {
        arenaPosition_ = node_->GetPosition();
        animController_ = node_->GetOrCreateComponent<AnimationController>();
        SetupPhysics();
    }
    Reset();
}

void Fighter::SetupPhysics()
{
    if (!node_) return;
    rigidBody_ = node_->GetOrCreateComponent<RigidBody>();
    if (rigidBody_)
    {
        rigidBody_->SetMass(80.0f);
        rigidBody_->SetUseGravity(true); // Gravity active like Ninja Sample
        rigidBody_->SetLinearDamping(10.0f);
        rigidBody_->SetAngularDamping(10.0f);
        rigidBody_->SetKinematic(false);
     rigidBody_->SetCollisionLayer(LAYER_FIGHTER);
     rigidBody_->SetCollisionMask(LAYER_WORLD | LAYER_ZOMBIE); // Collide only with world and zombie

        // NINJA SAMPLE RULE: Freeze all angular rotations (X, Y, Z rotation tipping over)
        // This forces the capsule to stay perfectly vertical on the ground grid at all times
        rigidBody_->SetAngularFactor(Vector3::ZERO);
    }
    collisionShape_ = node_->GetOrCreateComponent<CollisionShape>();
    if (collisionShape_) collisionShape_->SetCapsule(0.45f, 1.6f, Vector3(0.0f, 0.8f, 0.0f));
}

void Fighter::Reset()
{
    isSwinging_ = false;
    isBiteSynced_ = false;
    isLunging_ = false;          // <-- ADD
    lungeDistCovered_ = 0.0f;    // <-- ADD
    swingTimer_ = 0.0f;

    if (node_)
    {
        node_->SetPosition(arenaPosition_);
        node_->SetRotation(Quaternion::IDENTITY);
        if (rigidBody_)
        {
            rigidBody_->SetKinematic(false);
            rigidBody_->SetAngularFactor(Vector3::ZERO); // Re-enforce lock
            rigidBody_->SetPosition(arenaPosition_);
            rigidBody_->SetRotation(Quaternion::IDENTITY);
            rigidBody_->SetLinearVelocity(Vector3::ZERO);
            rigidBody_->ResetForces();
            rigidBody_->Activate();
        }
    }
    ResetMeshLocalTransform();
    if (animController_) animController_->StopAll();
}

void Fighter::ResetMeshLocalTransform()
{
    if (meshNode_)
    {
        meshNode_->SetRotation(Quaternion(0.0f, -90.0f, 0.0f));
        meshNode_->SetScale(5.0f / 9.0f);
        meshNode_->SetPosition(Vector3(0.0f, 0.05f, 0.0f));
    }
}

void Fighter::StartAttack()
{
    if (isSwinging_ || isBiteSynced_) return;
    if (!animController_) return;

    isSwinging_ = true;
    swingTimer_ = 0.0f;
    lungeDistCovered_ = 0.0f;    // <-- ADD

    // If no lunge target was fed in, don't lunge
    if (lungeTarget_.Equals(Vector3::ZERO))
        isLunging_ = false;

    if (rigidBody_) rigidBody_->SetUseGravity(false);   // <-- ADD

    animController_->StopAll();
    AnimationParameters params(context_, attackAnimPath_);
    params.looped_ = false;
    animController_->PlayNewExclusive(params);
    animController_->SetTime(attackAnimPath_, 0.0f);
    animController_->SetSpeed(attackAnimPath_, 1.0f);
}

void Fighter::FixedUpdate(float timeStep)
{
    // ---- ABSOLUTE ANTI-SINK: lock Y to floor every physics frame ----
    if (node_)
    {
        Vector3 pos = node_->GetPosition();
        if (std::abs(pos.y_ - arenaPosition_.y_) > 0.0001f)
        {
            pos.y_ = arenaPosition_.y_;
            node_->SetPosition(pos);
        }
    }
    if (rigidBody_)
    {
        Vector3 rbPos = rigidBody_->GetPosition();
        if (std::abs(rbPos.y_ - arenaPosition_.y_) > 0.0001f)
        {
            rbPos.y_ = arenaPosition_.y_;
            rigidBody_->SetPosition(rbPos);
        }
        Vector3 vel = rigidBody_->GetLinearVelocity();
        if (vel.y_ != 0.0f)
        {
            vel.y_ = 0.0f;
            rigidBody_->SetLinearVelocity(vel);
        }
    }

    if (isBiteSynced_) return;

    if (isSwinging_)
    {
        swingTimer_ += timeStep;

        // ---- PHYSICS STEP: natural walk-in, no teleport ----
        if (isLunging_ && rigidBody_ && swingTimer_ <= 0.45f)
        {
            Vector3 toTarget = lungeTarget_ - node_->GetPosition();
            toTarget.y_ = 0.0f;
            float distRemaining = toTarget.Length();

            if (distRemaining > 0.02f)
            {
                toTarget.Normalize();
                float speed = 1.2f;   // ~1.2 m/s — a single natural step
                rigidBody_->SetLinearVelocity(
                    Vector3(toTarget.x_ * speed, 0.0f, toTarget.z_ * speed));
            }
            else
            {
                rigidBody_->SetLinearVelocity(Vector3::ZERO);
                isLunging_ = false;
            }
        }
        else if (isLunging_)
        {
            rigidBody_->SetLinearVelocity(Vector3::ZERO);
            isLunging_ = false;
        }

            if (!animController_->IsPlaying(attackAnimPath_))
            {
                isSwinging_ = false;
                isLunging_ = false;
                lungeDistCovered_ = 0.0f;
                swingTimer_ = 0.0f;
                ResetMeshLocalTransform();

                if (rigidBody_)
                {
                    rigidBody_->SetUseGravity(true);   // <-- ADD
                    rigidBody_->SetPosition(arenaPosition_);
                    rigidBody_->SetLinearVelocity(Vector3::ZERO);
                    rigidBody_->Activate();
                }
            }
    }

    if (rigidBody_ && !isSwinging_)
    {
        Vector3 pos = rigidBody_->GetPosition();
        Vector3 drift = arenaPosition_ - pos;
        drift.y_ = 0.0f;
        if (drift.Length() > 0.001f)
        {
            rigidBody_->SetPosition(arenaPosition_);
            rigidBody_->SetLinearVelocity(Vector3::ZERO);
        }
        if (node_->GetPosition() != arenaPosition_) node_->SetPosition(arenaPosition_);
    }
}

void Fighter::ReparentWeapon(Node* meshNode)
{
    if (!meshNode) return;
    auto* animatedModel = meshNode->FindComponent<AnimatedModel>();
    if (!animatedModel) return;

    Skeleton& skeleton = animatedModel->GetSkeleton();
    Bone* rightHandBone = skeleton.GetBone("RightHand");
    if (rightHandBone && rightHandBone->node_)
    {
        ea::vector<Node*> allDescendants;
        ea::vector<Node*> stack = { meshNode };
        while (!stack.empty())
        {
            Node* current = stack.back();
            stack.pop_back();
            for (Node* child : current->GetChildren())
            {
                allDescendants.push_back(child);
                stack.push_back(child);
            }
        }
        for (Node* child : allDescendants)
        {
            auto* sm = child->GetComponent<StaticModel>();
            if (!sm) continue;
            Model* mdl = sm->GetModel();
            if (!mdl || std::strstr(mdl->GetName().c_str(), "BattleAxe") == nullptr) continue;

            Vector3 worldPos = child->GetWorldPosition();
            Quaternion worldRot = child->GetWorldRotation();
            child->SetParent(rightHandBone->node_);
            Quaternion handInv = rightHandBone->node_->GetWorldRotation().Inverse();
            child->SetPosition(handInv * (worldPos - rightHandBone->node_->GetWorldPosition()));
            child->SetRotation(handInv * worldRot);
        }
    }
}
"Fighter.h"
---------
#pragma once
#include <Urho3D/Scene/LogicComponent.h>
#include <Urho3D/Graphics/AnimationController.h>
#include <Urho3D/Physics/RigidBody.h>
#include <Urho3D/Physics/CollisionShape.h>
#include <Urho3D/Graphics/AnimatedModel.h>

using namespace Urho3D;

class Fighter : public LogicComponent
{
    URHO3D_OBJECT(Fighter, LogicComponent);

public:
    Fighter(Context* context);
    ~Fighter() override = default;

    static void RegisterObject(Context* context);

    void Initialize(Node* node, const ea::string& prefabPath);
    void Initialize(Node* node, const ea::string& idleAnimPath, const ea::string& attackAnimPath);
    void Reset();
    void StartAttack();
    void FixedUpdate(float timeStep) override;

    void SetBiteSynced(bool synced) { isBiteSynced_ = synced; }
    bool IsSwinging() const { return isSwinging_; }
    bool IsMidSwing() const { return isSwinging_ && (swingTimer_ >= 0.20f && swingTimer_ <= 0.60f); }
    Vector3 GetPosition() const { return node_ ? node_->GetPosition() : Vector3::ZERO; }
    Quaternion GetRotation() const { return node_ ? node_->GetRotation() : Quaternion::IDENTITY; }
    RigidBody* GetRigidBody() const { return rigidBody_; }
    void SetLungeTarget(const Vector3& target) { lungeTarget_ = target; isLunging_ = true; }
    void ClearLungeTarget() { lungeTarget_ = Vector3::ZERO; isLunging_ = false; }

private:
    void ReparentWeapon(Node* meshNode);
    void ResetMeshLocalTransform();
    void SetupPhysics();
    void SetupBoneColliders();

    SharedPtr<Node> node_;
    SharedPtr<Node> meshNode_;
    SharedPtr<AnimationController> animController_;
    SharedPtr<RigidBody> rigidBody_;
    SharedPtr<CollisionShape> collisionShape_;

    ea::string idleAnimPath_;
    ea::string attackAnimPath_;

    bool isSwinging_{false};
    bool isBiteSynced_{false};
    float swingTimer_{0.0f};
    Vector3 arenaPosition_{Vector3::ZERO};
    Vector3 lungeTarget_{Vector3::ZERO};
    bool isLunging_{false};
    static constexpr float MAX_LUNGE_DIST = 0.40f;   // total step length (tune me)
    float lungeDistCovered_ = 0.0f;
};
"main.cpp"
---------
#include <Urho3D/Urho3D.h>
#include <Urho3D/Engine/Engine.h>
#include <Urho3D/Engine/EngineDefs.h>
#include <Urho3D/Graphics/Camera.h>
#include <Urho3D/Graphics/Light.h>
#include <Urho3D/Graphics/Octree.h>
#include <Urho3D/Graphics/Renderer.h>
#include <Urho3D/Graphics/Viewport.h>
#include <Urho3D/Graphics/Zone.h>
#include <Urho3D/Graphics/DebugRenderer.h>
#include <Urho3D/Graphics/StaticModel.h>
#include <Urho3D/Graphics/Model.h>
#include <Urho3D/Graphics/Material.h>
#include <Urho3D/Container/Str.h>
#include <Urho3D/IO/Log.h>
#include <Urho3D/Resource/ResourceCache.h>
#include <Urho3D/Scene/Scene.h>
#include <Urho3D/Scene/PrefabResource.h>
#include <Urho3D/Input/Input.h>
#include <Urho3D/Graphics/GraphicsDefs.h>
#include <Urho3D/UI/Text.h>
#include <Urho3D/UI/UI.h>
#include <Urho3D/UI/Font.h>
#include <Urho3D/Core/Timer.h>
#include <Urho3D/Physics/PhysicsWorld.h>

// Include our new object-oriented components
#include "Fighter.h"
#include "Zombie.h"
#include "CombatSystem.h"

using namespace Urho3D;

int main(int argc, char** argv)
{
    SharedPtr<Context> context(new Context());
    SharedPtr<Engine> engine(new Engine(context));

    StringVariantMap engineParameters;
    engineParameters[EP_WINDOW_TITLE] = "Kilo-Code Battle Arena";
    engineParameters[EP_RESOURCE_PREFIX_PATHS] = "Data";
    engineParameters[EP_FULL_SCREEN] = false;
    engineParameters[EP_WINDOW_WIDTH] = 1152;
    engineParameters[EP_WINDOW_HEIGHT] = 864;
    engineParameters[EP_MULTI_SAMPLE] = 1;
    engineParameters[EP_VSYNC] = true;

    if (!engine->Initialize(engineParameters, {}))
    {
        URHO3D_LOGERROR("Could not initialise the engine.");
        return 1;
    }

    // REGISTER CUSTOM COMPONENT FACTORIES SO THE ENGINE CAN REFLECT THEM
    context->RegisterFactory<Fighter>();
    context->RegisterFactory<Zombie>();
    context->RegisterFactory<CombatSystem>();

    auto* renderer = engine->GetSubsystem<Renderer>();
    auto* cache    = engine->GetSubsystem<ResourceCache>();
    auto* input    = engine->GetSubsystem<Input>();
    auto* ui       = engine->GetSubsystem<UI>();
    auto* time     = engine->GetSubsystem<Time>();

    // ---- Diagnostics HUD Overlay ----
    auto* font = cache->GetResource<Font>("Fonts/Anonymous Pro.ttf");
    auto* debugText = ui->GetRoot()->CreateChild<Text>();
    if (font && debugText)
    {
        debugText->SetText("SPACEBAR: ATTACK / RESPAWN | W/S: ZOOM | A/D: ORBIT");
        debugText->SetFont(font, 16);
        debugText->SetColor(Color::GREEN);
        debugText->SetHorizontalAlignment(HA_CENTER);
        debugText->SetVerticalAlignment(VA_TOP);
    }

     // ---- World Scene Layout Initialization ----
     SharedPtr<Scene> scene(new Scene(context));
     scene->CreateComponent<Octree>();
     scene->CreateComponent<DebugRenderer>();
     scene->CreateComponent<PhysicsWorld>(); // Required for RigidBody components

    auto* z = scene->CreateComponent<Zone>();
    z->SetBoundingBox(BoundingBox(-1000.0f, 1000.0f));
    z->SetAmbientColor(Color(0.35f, 0.35f, 0.35f));
    z->SetFogColor(Color(0.55f, 0.55f, 0.55f));
    z->SetFogStart(150.0f);
    z->SetFogEnd(250.0f);

    Node* l = scene->CreateChild("KeyLight");
    l->SetDirection(Vector3(0.2f, -0.8f, 0.5f));
    auto* dl = l->CreateComponent<Light>();
    dl->SetLightType(LIGHT_DIRECTIONAL);
    dl->SetColor(Color(0.8f, 0.784f, 0.76f));
    dl->SetBrightness(0.72f);
    dl->SetCastShadows(false);

    Node* fillLightNode = scene->CreateChild("FillLight");
    fillLightNode->SetDirection(Vector3(-0.4f, -0.6f, -0.2f));
    auto* fillLight = fillLightNode->CreateComponent<Light>();
    fillLight->SetLightType(LIGHT_DIRECTIONAL);
    fillLight->SetColor(Color(0.4f, 0.4f, 0.44f));
    fillLight->SetBrightness(0.32f);

    auto* cameraNode = scene->CreateChild("Camera");
    cameraNode->SetPosition(Vector3(0.0f, 1.2f, -3.5f));
    cameraNode->LookAt(Vector3(0.0f, 0.6f, 1.6f));
    auto* camera = cameraNode->CreateComponent<Camera>();

    Node* camLightNode = scene->CreateChild("ViewerLight");
    camLightNode->SetPosition(cameraNode->GetPosition());
    camLightNode->SetRotation(cameraNode->GetRotation());
    auto* camLight = camLightNode->CreateComponent<Light>();
    camLight->SetLightType(LIGHT_SPOT);
    camLight->SetColor(Color(1.2f, 1.12f, 1.04f));
    camLight->SetBrightness(3.2f);
    camLight->SetRange(40.0f);
    camLight->SetFov(72.0f);
    camLight->SetCastShadows(false);

    SharedPtr<Viewport> viewport(new Viewport(context, scene, camera));
    viewport->SetDrawDebug(true);
    renderer->SetViewport(0, viewport);

    // =========================================================================
    // COMPONENT CREATION & BINDING POOL
    // =========================================================================

    // 1. Create and attach the Fighter component
    Node* axeManPivot = scene->CreateChild("AxeManPivot");
    axeManPivot->SetPosition(Vector3(0.59f, 0.0f, 1.6f));
    auto* fighterComponent = axeManPivot->CreateComponent<Fighter>();
    fighterComponent->Initialize(axeManPivot, "test.glb.d/Prefab.prefab");

    // 2. Create and attach the Zombie component
    Node* zombiePivot = scene->CreateChild("ZombiePivot");
    zombiePivot->SetPosition(Vector3(-2.0f, 0.0f, 1.6f));  // Moved closer to be visible on screen
    auto* zombieComponent = zombiePivot->CreateComponent<Zombie>();
    const ea::string zombiePrefabPath = "firing-running-didn'twork/test_out/test.gltf.d/Prefab.prefab";
    zombieComponent->Initialize(zombiePivot, zombiePrefabPath);

    // 3. Setup the Master Combat System Arbitrator Component
    Node* coreSystemNode = scene->CreateChild("CombatSystemNode");
    auto* combatSystem = coreSystemNode->CreateComponent<CombatSystem>();
    combatSystem->Initialize(fighterComponent, zombieComponent, debugText);

    // =========================================================================
    // MAIN LOOP
    // =========================================================================
      while (!engine->IsExiting())
      {
        engine->RunFrame();
        float dt = time->GetTimeStep();

        // --- ZOMBIE AUTOPSY: run every frame, but only log every 30th to save IO ---
        static int debugFrame = 0;
        debugFrame++;
        bool logNow = (debugFrame % 30 == 1);

        if (logNow && zombieComponent)
        {
            Node* zPivot = zombieComponent->GetNode();
            if (zPivot)
            {
                URHO3D_LOGINFOF("[Z-AUTOPSY] Pivot    pos=%s rot=%s",
                                zPivot->GetPosition().ToString().c_str(),
                                zPivot->GetRotation().ToString().c_str());

                Node* zMesh = zPivot->GetChild("ZombieMesh");
                if (zMesh)
                {
                    URHO3D_LOGINFOF("[Z-AUTOPSY] Mesh     worldPos=%s scale=%s",
                                    zMesh->GetWorldPosition().ToString().c_str(),
                                    zMesh->GetScale().ToString().c_str());

                    AnimatedModel* am = zMesh->FindComponent<AnimatedModel>();
                    if (!am) am = zPivot->FindComponent<AnimatedModel>(); // fallback
                    if (am)
                    {
                        BoundingBox bb = am->GetWorldBoundingBox();
                        URHO3D_LOGINFOF("[Z-AUTOPSY] WorldBB  min=%s max=%s size=%s",
                                        bb.min_.ToString().c_str(),
                                        bb.max_.ToString().c_str(),
                                        bb.Size().ToString().c_str());
                        URHO3D_LOGINFOF("[Z-AUTOPSY] Drawable enabled=%d castShadows=%d",
                                        am->IsEnabled(), am->GetCastShadows());
                    }
                    else
                    {
                        URHO3D_LOGWARNING("[Z-AUTOPSY] NO AnimatedModel found!");
                    }
                }
                else
                {
                    URHO3D_LOGWARNING("[Z-AUTOPSY] NO 'ZombieMesh' child found!");
                }

                RigidBody* zBody = zombieComponent->GetRigidBody();
                if (zBody)
                {
                    URHO3D_LOGINFOF("[Z-AUTOPSY] Physics  pos=%s vel=%s mass=%.1f",
                                    zBody->GetPosition().ToString().c_str(),
                                    zBody->GetLinearVelocity().ToString().c_str(),
                                    zBody->GetMass());
                }
            }
        }

        // --- VISUAL DEBUG DRAW ---
        auto* dbg = scene->GetComponent<DebugRenderer>();
        if (dbg)
        {
            dbg->SetEnabled(true); // make sure it's on

            if (zombieComponent && zombieComponent->GetNode())
            {
                Node* zPivot = zombieComponent->GetNode();
                // Red wire sphere at pivot
                dbg->AddSphere(Sphere(zPivot->GetWorldPosition(), 0.2f), Color::RED);

                Node* zMesh = zPivot->GetChild("ZombieMesh");
                if (zMesh)
                {
                    AnimatedModel* am = zMesh->FindComponent<AnimatedModel>();
                    if (am)
                    {
                        // Green wire bounding box of the actual model
                        dbg->AddBoundingBox(am->GetWorldBoundingBox(), Color::GREEN);
                    }
                }
            }

            // Also draw fighter for reference
            if (fighterComponent && fighterComponent->GetNode())
            {
                Node* fPivot = fighterComponent->GetNode();
                dbg->AddSphere(Sphere(fPivot->GetWorldPosition(), 0.2f), Color::BLUE);
                AnimatedModel* fam = fPivot->FindComponent<AnimatedModel>();
                if (!fam && fPivot->GetChild("AxeManMesh"))
                    fam = fPivot->GetChild("AxeManMesh")->FindComponent<AnimatedModel>();
                if (fam)
                    dbg->AddBoundingBox(fam->GetWorldBoundingBox(), Color::CYAN);
            }
        }

        if (input->GetKeyPress(KEY_SPACE))
        {
            if (combatSystem->IsArenaHitFrozen())
            {
                combatSystem->ResetSimulation();
            }
            else
            {
                combatSystem->HandleManualAttack();
            }
        }

        bool shrink = input->GetKeyPress(KEY_MINUS) || input->GetKeyPress(KEY_KP_MINUS);
        bool grow   = input->GetKeyPress(KEY_EQUALS) || input->GetKeyPress(KEY_KP_PLUS);
        if (shrink) combatSystem->GrowBloodWounds(-0.25f);
        if (grow)   combatSystem->GrowBloodWounds( 0.25f);

          combatSystem->UpdateCombat(dt);

        // Camera controls
        const Vector3 screenCenterAnchor(0.0f, 0.6f, 1.6f);
        Vector3 anchorToCamVector = cameraNode->GetPosition() - screenCenterAnchor;

        bool orbitLeft  = input->GetKeyDown(KEY_A) || input->GetKeyDown(KEY_Q);
        bool orbitRight = input->GetKeyDown(KEY_D) || input->GetKeyDown(KEY_E);

        if (orbitLeft || orbitRight)
        {
            float rotationDirection = orbitLeft ? -90.0f : 90.0f;
            rotationDirection *= dt;
            Quaternion yawOrbitRotation(rotationDirection, Vector3::UP);
            anchorToCamVector = yawOrbitRotation * anchorToCamVector;
            cameraNode->SetPosition(screenCenterAnchor + anchorToCamVector);
        }

        if (input->GetKeyDown(KEY_W)) cameraNode->Translate(Vector3::FORWARD * 3.0f * dt);
        if (input->GetKeyDown(KEY_S)) cameraNode->Translate(Vector3::BACK * 3.0f * dt);

        cameraNode->LookAt(screenCenterAnchor);

        camLightNode->SetPosition(cameraNode->GetPosition());
        camLightNode->SetRotation(cameraNode->GetRotation());

         if (input->GetKeyPress(KEY_ESCAPE)) engine->Exit();

     }

    renderer->SetViewport(0, nullptr);
    engine->Exit();

    return 0;
}
"Zombie.cpp"
---------
#include "Zombie.h"
#include "Fighter.h"
#include <Urho3D/IO/Log.h>
#include <Urho3D/Core/Context.h>
#include <Urho3D/Resource/ResourceCache.h>
#include <Urho3D/Scene/PrefabResource.h>
#include <Urho3D/Physics/PhysicsEvents.h>
#include <Urho3D/IO/MemoryBuffer.h>
#include <cmath>

Zombie::Zombie(Context* context) : LogicComponent(context)
    , onGround_(false)
    , wasOnGround_(false)
    , isSliding_(false)
    , inAirTime_(0.0f)
{
    SetUpdateEventMask(USE_FIXEDUPDATE);
}

void Zombie::RegisterObject(Context* context)
{
    context->RegisterFactory<Zombie>();
}

void Zombie::Initialize(Node* node, const ea::string& prefabPath)
{
    node_ = node;
    if (!node_) return;

    startPosition_ = node_->GetPosition();
    node_->SetRotation(Quaternion(0.0f, 90.0f, 0.0f));

    size_t lastSlash = prefabPath.find_last_of('/');
    runAnimPath_ = (lastSlash != ea::string::npos) ? prefabPath.substr(0, lastSlash) + "/Animations/mixamo.com.ani" : prefabPath;

    meshNode_ = node_->CreateChild("ZombieMesh");
    auto* cache = GetSubsystem<ResourceCache>();
    auto* prefab = cache->GetResource<PrefabResource>(prefabPath);
    if (prefab)
    {
        prefab->InstantiateReference(meshNode_);
        animController_ = meshNode_->FindComponent<AnimationController>();
        meshNode_->SetScale(75.0f);
        meshNode_->SetPosition(Vector3::ZERO);
        meshNode_->SetRotation(Quaternion::IDENTITY);
        SetupPhysics();
    }

    SubscribeToEvent(node_, E_NODECOLLISION, URHO3D_HANDLER(Zombie, HandleNodeCollision));

    Reset();
}

void Zombie::SetupPhysics()
{
    if (!node_) return;
    rigidBody_ = node_->GetOrCreateComponent<RigidBody>();
    if (rigidBody_)
    {
        rigidBody_->SetMass(60.0f);
        // CRITICAL FIX: Start with NO gravity until we verify ground exists
        // If ground plane exists in main.cpp, this works fine
        // If not, zombie won't fall forever
        rigidBody_->SetUseGravity(true);
        rigidBody_->SetLinearDamping(1.0f);   // Reduced damping for easier movement
        rigidBody_->SetAngularDamping(10.0f);
        rigidBody_->SetKinematic(false);
        rigidBody_->SetCollisionLayer(Z_LAYER_ZOMBIE);
        rigidBody_->SetCollisionMask(Z_LAYER_WORLD | Z_LAYER_FIGHTER);
        rigidBody_->SetAngularFactor(Vector3::ZERO);
        rigidBody_->SetRestitution(0.0f);
        rigidBody_->SetFriction(0.6f);        // Slightly reduced friction
    }
    collisionShape_ = node_->GetOrCreateComponent<CollisionShape>();
    if (collisionShape_)
        collisionShape_->SetCapsule(0.45f, 1.6f, Vector3(0.0f, 0.8f, 0.0f));
}

void Zombie::HandleNodeCollision(StringHash eventType, VariantMap& eventData)
{
    using namespace NodeCollision;

    RigidBody* otherBody = static_cast<RigidBody*>(eventData[P_OTHERBODY].GetPtr());
    if (!otherBody) return;

    // Accept any static/kinematic body as ground (layer 1 = world, or kinematic flag)
    bool isGround = (otherBody->GetCollisionLayer() == Z_LAYER_WORLD) ||
                    (otherBody->GetMass() == 0.0f && otherBody->IsKinematic());

    if (isGround)
    {
        VectorBuffer contacts(eventData[P_CONTACTS].GetBuffer());
        while (!contacts.IsEof())
        {
            Vector3 contactPosition = contacts.ReadVector3();
            Vector3 contactNormal = contacts.ReadVector3();
            contacts.ReadFloat(); // distance
            contacts.ReadFloat(); // impulse

            // Ground contact: contact point below our center AND normal pointing up
            if (contactPosition.y_ < node_->GetPosition().y_ + 0.5f && contactNormal.y_ > 0.5f)
            {
                onGround_ = true;
            }
        }
    }
}

void Zombie::FixedUpdate(float timeStep)
{
    // ---- ABSOLUTE ANTI-SINK: lock Y to floor every physics frame ----
    const float floorY = startPosition_.y_;
    if (node_)
    {
        Vector3 pos = node_->GetPosition();
        if (std::abs(pos.y_ - floorY) > 0.0001f)
        {
            pos.y_ = floorY;
            node_->SetPosition(pos);
        }
    }
    if (rigidBody_)
    {
        Vector3 rbPos = rigidBody_->GetPosition();
        if (std::abs(rbPos.y_ - floorY) > 0.0001f)
        {
            rbPos.y_ = floorY;
            rigidBody_->SetPosition(rbPos);
        }
        Vector3 vel = rigidBody_->GetLinearVelocity();
        if (vel.y_ != 0.0f)
        {
            vel.y_ = 0.0f;
            rigidBody_->SetLinearVelocity(vel);
        }
    }

    if (isBiteSynced_ || health_ <= 0 || isFrozen_ || isNeckBiting_) return;

    wasOnGround_ = onGround_;
    onGround_ = false;
    isSliding_ = false;

    // FALLBACK: If y position is near zero and we have no ground collision,
    // assume we're on implicit ground (for arenas without explicit ground plane)
    Vector3 pos = node_->GetPosition();
    if (pos.y_ <= 0.05f && !wasOnGround_)
    {
        // Snap to ground level and mark as grounded
        pos.y_ = 0.0f;
        node_->SetPosition(pos);
        if (rigidBody_)
        {
            Vector3 vel = rigidBody_->GetLinearVelocity();
            if (vel.y_ < 0) vel.y_ = 0;
            rigidBody_->SetLinearVelocity(vel);
            rigidBody_->SetPosition(pos);
        }
        onGround_ = true;
        wasOnGround_ = true;
    }

    UpdateMovement(timeStep);
    UpdateAnimation();
}

void Zombie::UpdateMovement(float timeStep)
{
    if (!rigidBody_ || !node_) return;

    Vector3 vel = rigidBody_->GetLinearVelocity();

    if (!isContactFrozen_)
    {
        // === WALKING TOWARD FIGHTER (+X) ===
        // Use ApplyForce for continuous force (not ApplyImpulse which is instantaneous)
        // Force = mass * acceleration. For 60kg body, 60N = 1 m/s² acceleration
        // We want to reach WALK_SPEED over ~0.5s, so accel = 1.8/0.5 = 3.6 m/s²
        // Force = 60 * 3.6 = 216 N
        static constexpr float MOVE_FORCE = 1620.0f;  // Strong enough to overcome friction

        if (wasOnGround_)
        {
            // Apply continuous force in +X direction
            rigidBody_->ApplyForce(Vector3(MOVE_FORCE, 0.0f, 0.0f));
            inAirTime_ = 0.0f;
        }
        else
        {
            // In air — minimal horizontal control
            rigidBody_->ApplyForce(Vector3(MOVE_FORCE * 0.2f, 0.0f, 0.0f));
            inAirTime_ += timeStep;
        }

        // Cap max speed
        Vector3 horizontalVel(vel.x_, 0.0f, vel.z_);
        float hSpeed = horizontalVel.Length();
        if (hSpeed > WALK_SPEED)
        {
            horizontalVel.Normalize();
            horizontalVel *= WALK_SPEED;
            rigidBody_->SetLinearVelocity(Vector3(horizontalVel.x_, vel.y_, horizontalVel.z_));
        }

        // Damping only when not actively pushing (creates natural coast-to-stop)
        // But not too much or we can't start moving
        if (hSpeed > WALK_SPEED * 0.1f)
        {
            Vector3 dampingForce(-DAMPING_FORCE * vel.x_ * 0.5f, 0.0f, -DAMPING_FORCE * vel.z_ * 0.5f);
            rigidBody_->ApplyForce(dampingForce);
        }

        // Zero tiny Y velocity when grounded to prevent micro-bouncing
        if (std::abs(vel.y_) < 0.05f && wasOnGround_)
        {
            vel.y_ = 0.0f;
            rigidBody_->SetLinearVelocity(vel);
        }
    }
    else
    {
        // CONTACT FROZEN — full stop
        rigidBody_->SetLinearVelocity(Vector3::ZERO);
        rigidBody_->SetAngularVelocity(Vector3::ZERO);
    }
}

void Zombie::UpdateAnimation()
{
    if (!animController_) return;

    if (isContactFrozen_ || isFrozen_)
    {
        if (!isFrozen_)
            animController_->SetSpeed(runAnimPath_, 0.0f);
    }
    else if (rigidBody_)
    {
        Vector3 vel = rigidBody_->GetLinearVelocity();
        float speed = Vector3(vel.x_, 0.0f, vel.z_).Length();

        // Sync animation to actual physics speed
        // At WALK_SPEED (1.8), anim now plays at 2.5x base speed (5x higher than before)
        float animSpeed = (speed / WALK_SPEED) * 12.5f;   // <-- WAS 0.5f
        animSpeed = Clamp(animSpeed, 0.0f, 5.0f);        // <-- WAS 1.0f
        animController_->SetSpeed(runAnimPath_, animSpeed);

        // DEBUG: Log speed occasionally
        static int frame = 0;
        if (++frame % 60 == 0)
        {
            URHO3D_LOGINFOF("[ZOMBIE] speed=%.3f animSpeed=%.3f onGround=%d pos=%s",
                speed, animSpeed, wasOnGround_,
                node_->GetPosition().ToString().c_str());
        }
    }
}

void Zombie::CheckCollision(Fighter* fighter)
{
    if (!node_ || !fighter || isFrozen_ || health_ <= 0 || isNeckBiting_ || isBiteSynced_) return;

    Vector3 myPos = rigidBody_ ? rigidBody_->GetPosition() : node_->GetPosition();
    Vector3 targetPos = fighter->GetPosition();
    float boundaryX = targetPos.x_ - STOP_THRESHOLD;
    const float floorY = startPosition_.y_;

    if (isContactFrozen_)
    {
        // Always snap to the current boundary (follows debug offset immediately)
        Vector3 clampedPos(boundaryX, floorY, myPos.z_);
        node_->SetPosition(clampedPos);
        rigidBody_->SetPosition(clampedPos);
        rigidBody_->SetLinearVelocity(Vector3::ZERO);
        return;
    }

    if (myPos.x_ >= boundaryX)
    {
        Vector3 stopPos(boundaryX, floorY, myPos.z_);
        node_->SetPosition(stopPos);
        rigidBody_->SetPosition(stopPos);
        rigidBody_->SetLinearVelocity(Vector3::ZERO);
        rigidBody_->SetAngularVelocity(Vector3::ZERO);
        isContactFrozen_ = true;
    }
}

void Zombie::HandleHit(Fighter* fighter)
{
    if (hasBeenHit_) return;
    hasBeenHit_ = true;
    isFrozen_ = true;
    if (animController_) animController_->StopAll();
    if (rigidBody_)
    {
        rigidBody_->SetLinearVelocity(Vector3::ZERO);
        rigidBody_->SetAngularVelocity(Vector3::ZERO);
        rigidBody_->SetKinematic(true);           // absolute lock
        Vector3 pos = rigidBody_->GetPosition();
        pos.y_ = startPosition_.y_;
        rigidBody_->SetPosition(pos);
    }
    if (node_)
    {
        Vector3 pos = node_->GetPosition();
        pos.y_ = startPosition_.y_;
        node_->SetPosition(pos);
    }
}

void Zombie::ApplyBoundaryFreeze()
{
    if (isContactFrozen_) return;
    isContactFrozen_ = true;
    if (rigidBody_) rigidBody_->SetLinearVelocity(Vector3::ZERO);
}

void Zombie::TriggerNeckBite()
{
    if (isNeckBiting_) return;
    isNeckBiting_ = true;
    if (rigidBody_) rigidBody_->SetLinearVelocity(Vector3::ZERO);
}

void Zombie::StartNeckBite(Node* targetNode)
{
    TriggerNeckBite();
}

void Zombie::Reset()
{
    isFrozen_ = false;
    isContactFrozen_ = false;
    isNeckBiting_ = false;
    isBiteSynced_ = false;
    hasBeenHit_ = false;
    health_ = maxHealth_;
    onGround_ = false;
    wasOnGround_ = false;
    isSliding_ = false;
    inAirTime_ = 0.0f;

    if (node_)
    {
        node_->SetPosition(startPosition_);
        node_->SetRotation(Quaternion(0.0f, 90.0f, 0.0f));
        if (rigidBody_)
        {
            rigidBody_->SetKinematic(false);
            rigidBody_->SetAngularFactor(Vector3::ZERO);
            rigidBody_->SetPosition(startPosition_);
            rigidBody_->SetRotation(Quaternion(0.0f, 90.0f, 0.0f));
            rigidBody_->SetLinearVelocity(Vector3::ZERO);
            rigidBody_->SetAngularVelocity(Vector3::ZERO);
            rigidBody_->ResetForces();
            rigidBody_->Activate();
        }
    }

    if (animController_ && !runAnimPath_.empty())
    {
        animController_->StopAll();
        AnimationParameters params(context_, runAnimPath_);
        params.looped_ = true;
        animController_->PlayNewExclusive(params);
        animController_->SetTime(runAnimPath_, 0.0f);
        animController_->SetSpeed(runAnimPath_, 150.0f);   // <-- WAS 0.5f
    }
}
"Zombie.h"
---------
#pragma once
#include <Urho3D/Scene/LogicComponent.h>
#include <Urho3D/Graphics/AnimationController.h>
#include <Urho3D/Physics/RigidBody.h>
#include <Urho3D/Physics/CollisionShape.h>
#include <Urho3D/Graphics/AnimatedModel.h>

using namespace Urho3D;

class Fighter;

// Physics collision layers
enum ZombieCollisionLayer {
    Z_LAYER_WORLD = 1,
    Z_LAYER_FIGHTER = 2,
    Z_LAYER_ZOMBIE = 4
};

class Zombie : public LogicComponent
{
    URHO3D_OBJECT(Zombie, LogicComponent);

public:
     Zombie(Context* context);
     ~Zombie() override = default;

     static void RegisterObject(Context* context);

     void Initialize(Node* node, const ea::string& prefabPath);
     void Reset();
     void FixedUpdate(float timeStep) override;

     void TriggerNeckBite();
     void StartNeckBite(Node* targetNode);
     void ApplyBoundaryFreeze();
     void CheckCollision(Fighter* fighter);
     void HandleHit(Fighter* fighter);

     void SetBiteSynced(bool synced) { isBiteSynced_ = synced; }
     bool IsFrozen() const { return isFrozen_; }
     bool IsContactFrozen() const { return isContactFrozen_; }
     bool IsNeckBiting() const { return isNeckBiting_; }
     Vector3 GetPosition() const { return node_ ? node_->GetPosition() : Vector3::ZERO; }
     RigidBody* GetRigidBody() const { return rigidBody_; }

     // NINJA-STYLE: Ground contact state for proper walk animation blending
     bool IsOnGround() const { return onGround_; }

private:
     void SetupPhysics();
     void UpdateMovement(float timeStep);
     void UpdateAnimation();
     void HandleNodeCollision(StringHash eventType, VariantMap& eventData);

     SharedPtr<Node> node_;
     SharedPtr<Node> meshNode_;
     SharedPtr<AnimationController> animController_;
     SharedPtr<RigidBody> rigidBody_;
     SharedPtr<CollisionShape> collisionShape_;

     ea::string runAnimPath_;
     Vector3 startPosition_{-4.5f, 0.0f, 1.6f};

     bool isFrozen_{false};
     bool isContactFrozen_{false};
     bool isNeckBiting_{false};
     bool isBiteSynced_{false};

     float health_{100.0f};
     const float maxHealth_{100.0f};

     // NINJA-STYLE: Movement constants (much slower than before)
     static constexpr float WALK_FORCE = 3.5f;        // Gentle push
     static constexpr float WALK_SPEED = 16.2f;        // Max walk speed
     static constexpr float DAMPING_FORCE = 12.0f;    // Friction when not pushing
     static constexpr float STOP_THRESHOLD = 0.94f;   // 1.2f - 0.26f permanent offset

     bool hasBeenHit_ = false;

     // NINJA-STYLE: Ground contact tracking
     bool onGround_{false};
     bool wasOnGround_{false};
     bool isSliding_{false};
     float inAirTime_{0.0f};
};
```